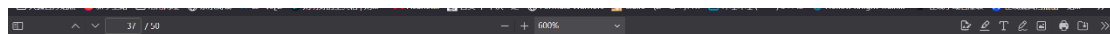


一、Misc(杂项)



最简单的一题，给了个乱码文档，翻了翻就看到不一样的地方，复制后 base64 解密后就是 flag

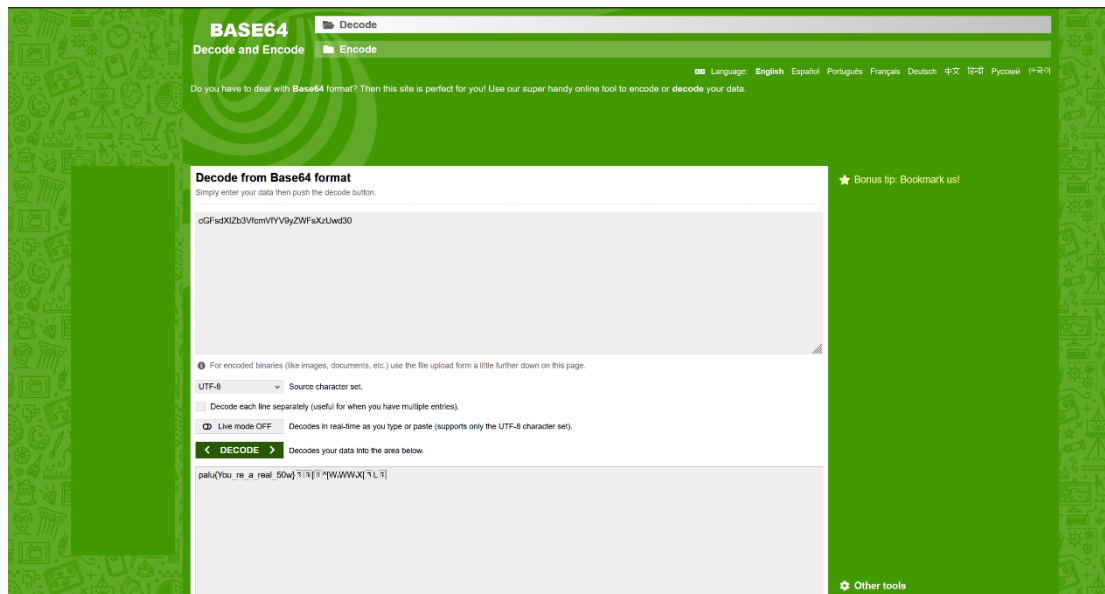


BnUbvqz8i23G8IYA6gNmcyG6uKo6awrRGm5

c8jwLsFccIdN8=vfmlYCnhP2U4k4j6WWI9r

cGFSqXwE53q32mVfWx9W8Wf6XzYr4M0SeTE

k4V=ααskT.nRrX0αn9azPiS8/8T9Xi115Knα1



题目附件:

[1970-01-01 08:00:00] System boot sequence initiated

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 00000000fe ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 10000000ef ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 20000000e2 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 30000000fb ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 40000000f5 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 50000000da ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 60000000e6 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 70000000e7 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 80000000fd ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 90000000d1 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 100000000e7 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 110000000fd ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 120000000d1 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 130000000cf ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 140000000d1 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 150000000dd ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 160000000ef ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 170000000e3 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 180000000fe ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 190000000e2 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 200000000eb ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 210000000d1 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 220000000c8 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 230000000e2 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 240000000ef ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 250000000e9 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 260000000d1 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 270000000cd ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 280000000e6 ns

[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 290000000ef ns

```
[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 300000000e0 ns
[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 310000000e9 ns
[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 320000000eb ns
[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 330000000d1 ns
[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 340000000c3 ns
[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 350000000eb ns
[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 360000000af ns
[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 370000000af ns
[1970-01-01 08:00:00] SYSTEM ALERT: Time anomaly detected at 380000000f3 ns
[1970-01-01 08:00:00] System entering chronostasis mode
```

解题脚本：

日志内容（直接粘贴到 logs 变量中，或从文件读取）

```
logs = ""
```

```
.....
```

步骤 1: 提取所有末尾的十六进制数

```
hex_bytes = []
```

```
for line in logs.split("\n"):
```

```
    if "SYSTEM ALERT" in line:
```

```
        # 提取最后两位十六进制字符（例如 "fe"）
```

```
        hex_part = line.split()[-2][-2:] # 倒数第二个词，取最后两位
```

```
        hex_bytes.append(hex_part)
```

步骤 2: 转换为字节并异或 0x8E

```
decrypted = []
```

```
for hb in hex_bytes:
```

```
    byte = int(hb, 16)          # 十六进制字符串转整数（例如 "fe" → 254）
```

```
decrypted_byte = byte ^ 0x8E # 异或解密
```

```
decrypted.append(decrypted_byte)
```

```
# 步骤 3: 转换为 ASCII 字符串
```

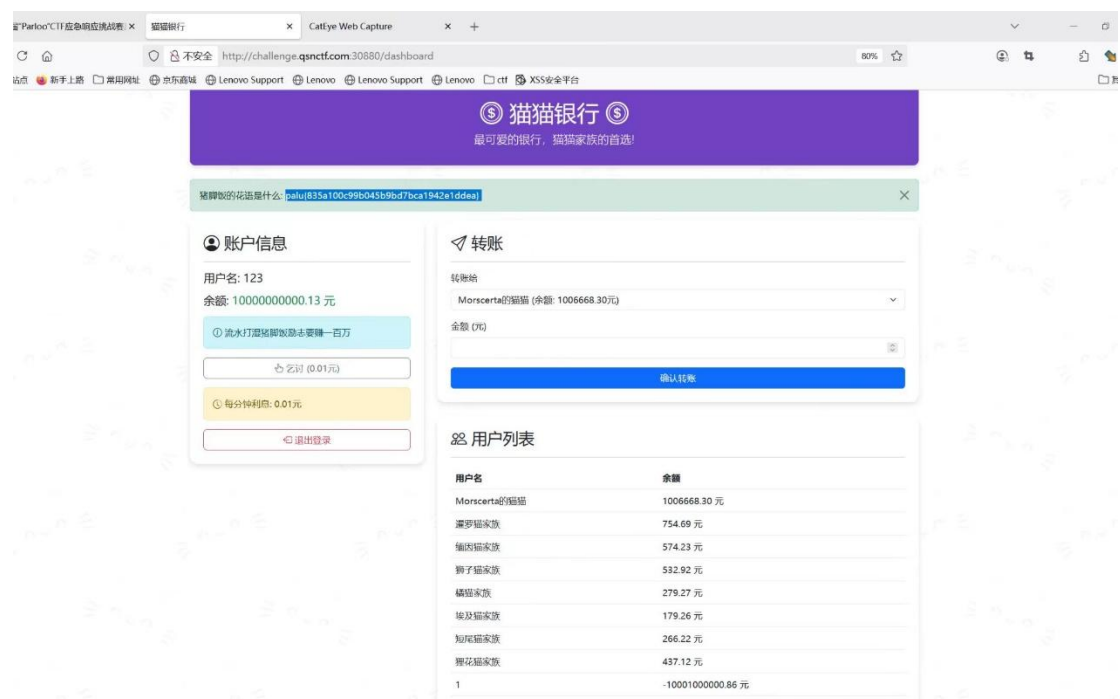
```
flag_bytes = bytes(decrypted)
```

```
flag = flag_bytes.decode("ascii")
```

```
print("Flag:", flag)
```

二、web





这题转 100 万就有 flag，FlowerC 做的，就截了一张图

三、Crypto(密码学)



题目：

题解：

```
import math

from Crypto.Util.number import long_to_bytes

# 从 output.txt 中获取的值

n =
12599781634575309604886589113907328689814346116951485805023283765790628
98408979740683911066089020829601710838177855327021582985896009478346994
94234633846206712414663927142998976208173208829799860130354978308649020
81588626245386519686739010503866650601772071227235941758667191706032389
1124382072599746305448903

# 填入 n

e = 65537

c =
16076213508704830809521504161524867240789661063230251272973700316524961
51184211006654774381216081334169128689580083039541305250251645181570561
04474848801125489343119145597766331407628639458190544324923923154911097
45915225117227073045171062365772401296382778452901831550773993089344837
645958797206220200272941

# 填入 c

s
=3593556926727214636844151259215348641924464903562364390298522081594019
83581460245903003940599093701158580912175977740104939386744727468283525
95432824315405933241792789402041405932624651226442192749572918686958461
02998824439687536129578510335674575630449746656734279632933115056077705
25882946380694888364197442972414091277296155446685471015803334205633184
86256358906310909703237944327684178950282413703357020770127158209107658
40700748956338898058263215912062186916533392166137799797033440778658102
42786982314187561067870580543557134723064097722606191177255618893508624
14726861327985706773512963177174611689685575805282

# 填入 s

# 计算 p 和 q

#  $p^2 + q^2 = \text{sqrt}(s + 2n^2)$ 
```

```
p2_plus_q2 = math.isqrt(s + 2 * n * n)
# p + q = sqrt(p2_plus_q2 + 2n)
p_plus_q = math.isqrt(p2_plus_q2 + 2 * n)
# 解方程  $x^2 - (p+q)x + n = 0$ 
D = p_plus_q * p_plus_q - 4 * n
p = (p_plus_q + math.isqrt(D)) // 2
q = p_plus_q - p
# 验证 p 和 q
assert p * q == n
# 计算私钥 d
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
# 解密密文
m = pow(c, d, n)
flag = long_to_bytes(m)
print(flag.decode())
```




```
import math

from Crypto.Util.number import long_to_bytes

# 从 output.txt 中获取的值

n =
12599781634575309604886589113907328689814346116951485805023283765790628
98408979740683911066089020829601710838177855327021582985896009478346994
94234633846206712414663927142998976208173208829799860130354978308649020
81588626245386519686739010503866650601772071227235941758667191706032389
1124382072599746305448903

# 填入 n

e = 65537

c =
16076213508704830809521504161524867240789661063230251272973700316524961
51184211006654774381216081334169128689580083039541305250251645181570561
04474848801125489343119145597766331407628639458190544324923923154911097
45915225117227073045171062365772401296382778452901831550773993089344837
645958797206220200272941
```

```
# 填入 c
```

```
s
```

```
=3593556926727214636844151259215348641924464903562364390298522081594019
83581460245903003940599093701158580912175977740104939386744727468283525
95432824315405933241792789402041405932624651226442192749572918686958461
02998824439687536129578510335674575630449746656734279632933115056077705
25882946380694888364197442972414091277296155446685471015803334205633184
86256358906310909703237944327684178950282413703357020770127158209107658
40700748956338898058263215912062186916533392166137799797033440778658102
42786982314187561067870580543557134723064097722606191177255618893508624
14726861327985706773512963177174611689685575805282
```

```
# 填入 s
```

```
# 计算 p 和 q
```

```
#  $p^2 + q^2 = \text{sqrt}(s + 2n^2)$ 
```

```
p2_plus_q2 = math.isqrt(s + 2 * n * n)
```

```
#  $p + q = \text{sqrt}(p2\_plus\_q2 + 2n)$ 
```

```
p_plus_q = math.isqrt(p2_plus_q2 + 2 * n)
```

```
# 解方程  $x^2 - (p+q)x + n = 0$ 
```

```
D = p_plus_q * p_plus_q - 4 * n
```

```
p = (p_plus_q + math.isqrt(D)) // 2
```

```
q = p_plus_q - p
```

```
# 验证 p 和 q
```

```
assert p * q == n
```

```
# 计算私钥 d
```

```
phi = (p - 1) * (q - 1)
```

```
d = pow(e, -1, phi)
```

```
# 解密密文
```

```
m = pow(c, d, n)
```

```
flag = long_to_bytes(m)
```

```
print(flag.decode())
```

四、Reverse(逆向)



```
encrypted = "qcoq~Vh{e~bccocH^@Lgt{gtlg" # 确保 encrypted 正确定义
```

```
# 尝试不同的常数 c，直到解密结果以 "flag{" 开头
```

```
for c in range(256):
```

```
    original = []
```

```
    for i in range(len(encrypted)):
```

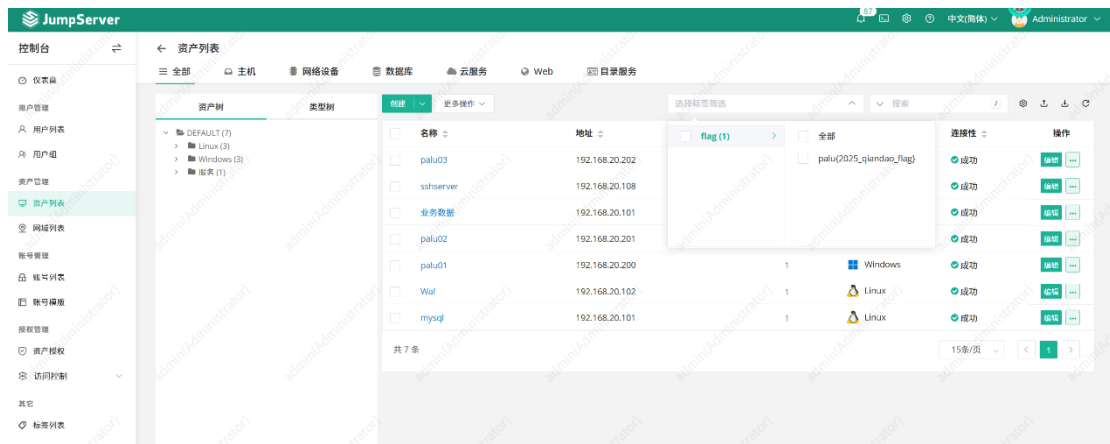
```
        key = i + c
```

```
encrypted_char = ord(encrypted[i])  
original_char = encrypted_char ^ key  
original.append(chr(original_char))  
  
original_str = ''.join(original)  
  
if original_str.startswith("palu{"): # 修正为 "flag{"  
    print(f"Found c = {c}: {original_str}")  
    break
```

五、应急响应



题目是堡垒机中的 flag



题目是寻找反序列化漏洞的端口作为 flag 进行提交

动作	攻击地址	攻击类型	源 IP	时间
未识别	http://192.168.20.102:9999/node/?_format=hal_json	反序列化	192.168.20.1 局域网	2025-05-14 00:13:03

100 条每页, 共 1 条

< 1 > 跳至 / 1 页